

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №3

«Алгоритм Дейкстры»

Выполнил:

студент группы 3824Б1ФИ2

Ненев А.Е.

Проверил:

Ивлев А.Д.

Нижний Новгород

2026

Содержание

Введение	3
1. Постановка задачи	4
2. Руководство пользователя	5
3. Руководство программиста	7
3.1. Описание структуры программы	7
3.2. Описание структур данных	9
3.3. Описание алгоритмов	12
4. Результаты экспериментов и тестирования	15
Заключение	19
Литература	20

Введение

Алгоритм Дейкстры решает задачу поиска кратчайших путей от одной вершины графа до всех остальных. Он широко применяется в навигаторах, маршрутизации сетей и логистике. Скорость работы алгоритма сильно зависит от того, как устроен граф и какая структура данных используется для выбора кратчайшего пути.

В данной работе сравниваются два способа хранения графа: матрица смежности и список смежности. Также сравниваются два типа куч (очереди с приоритетом): бинарная куча и биномиальная куча. Комбинируя эти варианты, получаем четыре различные реализации алгоритма Дейкстры.

1. Постановка задачи

Цель работы — разработать программу, которая позволяет осуществлять следующие действия:

- 1) Задания графа в ручном и автоматическом (случайная генерация) режиме.
- 2) Алгоритм Дейкстры с использованием приоритетной очереди, написанной на следующий структурах данных: бинарная куча и сложная куча.

Работоспособность программы необходимо проверить с помощью Google Test-ов.

Кроме того, необходимо разработать пользовательское консольное приложение.

Особенности реализации:

1. Необходимо реализовать любых два способа хранения графов
2. Должна быть проверка на односвязность графа, и удовлетворение условиям алгоритма Дейкстры.
3. Ожидается обработка только неориентированных графов
4. В качестве кучи нужно реализовать биномиальную и бинарную кучу
5. Алгоритм Дейкстры считает и сохраняет кратчайшие пути до всех точек.
6. Пользователь может задавать начальную точку для всего алгоритма.
7. После работы алгоритма пользователь может запросить кратчайший путь и его стоимость до любой точки.
8. Пользователь может запросить вывод графа на экран, в любом удобном для вас виде.

Эксперименты:

Цель: сравнить эффективность структур хранения графов и куч относительно их использования в алгоритме Дейкстры.

Методика: для каждой комбинации структуры хранения графа и кучи необходимо:

- Сгенерировать наборы случайных графов с различными количествами вершин и плотностью. Наборы графов одинаковы для всех экспериментов.
- Для каждого графа выполнить алгоритм Дейкстры от одной и той же случайно выбранной стартовой вершины при всех экспериментах.
- Для каждого запуска зафиксировать время выполнения алгоритма.

2. Руководство пользователя

Главное меню:

1. Change graph type (Matrix/List)
2. Change heap type (Binary/Binomial)
3. Create new empty graph
4. Generate random connected graph
5. Add edge
6. Run Dijkstra algorithm
7. Query shortest path to vertex
8. Print graph
0. Exit

Описание команд:

1. Выбор типа графа

Позволяет переключиться между двумя представлениями графа:

Matrix (матрица смежности)

List (список смежности)

После смены типа графа предыдущий граф удаляется.

2. Выбор типа кучи

Позволяет выбрать реализацию приоритетной очереди:

Binary (бинарная куча) - реализация на массиве

Binomial (биномиальная куча) - реализация на основе биномиальных деревьев

3. Создание пустого графа

Создает новый пустой граф с указанным количеством вершин.

Пример ввода: 10 (создать граф из 10 вершин)

4. Генерация случайного графа

Создает случайный связный взвешенный граф. Запрашиваются параметры:

Количество вершин — целое положительное число

Плотность рёбер — число от 0.0 до 1.0 (например, 0.4 означает, что ребро существует с вероятностью 40%)

Максимальный вес — вещественное число

5. Добавление ребра вручную

Добавляет ребро между двумя вершинами. Запрашиваются параметры:

От вершины — номер начальной вершины (целое число, начиная с 1)

До вершины — номер конечной вершины

Вес ребра — вещественное число (не может быть отрицательным)

6. Запуск алгоритма Дейкстры

Запускает алгоритм поиска кратчайших путей от указанной вершины. Запрашивается:

Стартовая вершина — номер вершины (целое число, начиная с 1)

После успешного выполнения программа сообщает о завершении работы алгоритма.

7. Запрос кратчайшего пути

Позволяет узнать кратчайшее расстояние от последней использованной стартовой вершины до выбранной вершины. Запрашивается:

Целевая вершина — номер вершины (целое число, начиная с 1)

8. Вывод графа на экран

Отображает текущее состояние графа. Для каждой вершины выводятся её соседи и веса рёбер.

0. Выход из программы

Завершает работу программы.

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из следующих модулей:

Модуль `triangle_matrix.h`

Взят из одной из прошлых лабораторных работ в почти неизменном виде, причем оттуда исключены алгебраические операции над векторами и матрицами, поскольку в них нет необходимости. Реализует шаблонный класс `TMatrix` для хранения верхнетреугольной матрицы. Используется для компактного хранения неориентированных графов в матричном представлении. Модуль также включает шаблонный класс `TVector` для работы с динамическими массивами с настраиваемым начальным индексом.

Модуль `vertex_distance.h`

Содержит структуру `Edge` для хранения информации о ребре (номер вершины и вес), а также структуру `VertexDistance` для хранения вершин с расстоянием при работе алгоритма Дейкстры.

Модуль `binary_heap.h`

Реализует шаблонный класс `BinaryHeap` — бинарную кучу (приоритетную очередь) на основе массива. Используется в алгоритме Дейкстры для эффективного извлечения вершины с минимальным расстоянием.

Модуль `binomial_heap.h`

Реализует шаблонный класс `BinomialHeap` — биномиальную кучу, состоящую из биномиальных деревьев. Включает структуру `Node` для представления узла кучи. Поддерживает операцию слияния куч за логарифмическое время.

Модуль `graph.h`

Содержит класс `Graph` — реализацию графа на основе матрицы смежности (с использованием `TMatrix`). Предоставляет методы для добавления/удаления рёбер и вершин, генерации случайных графов, а также реализации алгоритма Дейкстры с бинарной и биномиальной кучей.

Модуль `adjacency_list.h`

Содержит класс `AdjacencyList` — реализацию графа на основе списков смежности (вектор векторов структур `Edge`). Предоставляет аналогичный интерфейс для работы с графом и реализации алгоритма Дейкстры.

Модуль `app.cpp`

Содержит класс `GraphApplication`, реализующий пользовательский интерфейс (консольное меню). Обеспечивает взаимодействие пользователя с программой: выбор типа графа, типа кучи, генерацию графов, запуск алгоритмов и вывод результатов.

Модуль graph_tests.cpp

Содержит набор тестов на базе фреймворка Google Test. Тестирует корректность работы алгоритмов Дейкстры на фиксированных графах, производительность различных комбинаций структур, а также функциональность бинарной и биномиальной куч.

Модуль Experiments.cpp

Содержит эксперименты, оценивающие время выполнения алгоритма Дейкстры на сгенерированных графах на разном количестве вершин и с разной плотностью. При этом время оценивается для обоих способов хранения графа, как на матрице смежности, так и на списках смежности для алгоритма Дейкстры на основе бинарной и биномиальной куч.

3.2. Описание структур данных

Структура Edge

- **vertex_number_** - номер смежной вершины
- **distance_** - вес ребра (вещественное число)

Назначение: хранение информации о ребре графа

Структура VertexDistance

- **vertex** - номер вершины
- **distance** - расстояние до вершины
- **operator<, operator>, operator<=, operator>=** - сравнение по расстоянию

Назначение: использование в приоритетной очереди алгоритма Дейкстры

Структура Node

- **key** - значение узла
- **parent** - указатель на родителя
- **left_child** - указатель на левого ребёнка
- **sibling** - указатель на следующий узел в списке
- **degree** - степень узла

Назначение: представление узла биномиальной кучи

Класс TVector

Методы:

- **TVector(size, start_index)** - конструктор, создаёт массив заданного размера
- **TVector(other)** - конструктор копирования
- **~TVector()** - деструктор, освобождает память
- **GetSize(), GetStartIndex()** - возвращают размер и начальный индекс
- **operator[]** - доступ к элементу по индексу
- **operator==, operator!=** - сравнение векторов
- **operator=** - присваивание копированием

Назначение: динамический массив с настраиваемым начальным индексом

Класс TMatrix наследуется от TVector<TVector<ValType>>

Методы:

- **TMatrix(size)** - конструктор, создаёт квадратную матрицу
- **TMatrix(other)** - конструктор копирования
- **operator==, operator!=** - сравнение матриц
- **operator=** - присваивание

Назначение: верхнетреугольная матрица для хранения графа

Класс BinaryHeap

Методы:

- **BinaryHeap()** - конструктор по умолчанию
- **Insert(key)** - вставка элемента
- **ExtractMin()** - извлечение минимального элемента
- **IsEmpty()** - проверка на пустоту
- **SiftUp(i)** - просеивание вверх

- **SiftDown(i)** - просеивание вниз
- **Print()** - вывод кучи на экран

Назначение: бинарная куча (приоритетная очередь)

Класс BinomialHeap

Методы:

- **BinomialHeap()** - конструктор по умолчанию
- **BinomialHeap(key)** - конструктор с одним элементом
- **~BinomialHeap()** - деструктор
- **Merge(other)** - слияние двух куч
- **Insert(key)** - вставка элемента
- **ExtractMin()** - извлечение минимального элемента
- **DecreaseKey(node, key)** - уменьшение ключа узла
- **Delete(node)** — удаление узла
- **GetMin()** - получение минимального узла
- **IsEmpty()** - проверка на пустоту

Назначение: биномиальная куча (приоритетная очередь)

Класс Graph

Методы:

- **Graph()** - конструктор по умолчанию
- **Graph(vertices_number)** - конструктор с заданным размером
- **AddEdge(from, to, weight)** - добавление ребра
- **EraseRecord(from, to)** - удаление ребра
- **EraseVertex(vertex_number)** - удаление вершины
- **GetVerticesCount()** - получение количества вершин
- **GetEdgesCount()** - получение количества рёбер
- **HasEdge(from, to)** - проверка наличия ребра
- **GetEdgeWeight(from, to)** - получение веса ребра
- **Clear()** - очистка графа
- **IsEmpty()** - проверка на пустоту
- **MakeConnected(default_weight)** - достройка до связного
- **GenerateRandomConnectedGraph(vertices, density, maxWeight)** - генерация случайного графа
- **GetNeighbors(vertex)** - получение списка соседей
- **Dijkstra(start_vertex)** - алгоритм Дейкстры с бинарной кучей
- **DijkstraBinomial(start_vertex)** - алгоритм Дейкстры с биномиальной кучей
- **IsConnected()** - проверка связности
- **IsValidForDijkstra()** - проверка отсутствия отрицательных весов

Назначение: граф на матрице смежности

Класс AdjacencyList

Методы:

- **AdjacencyList()** - конструктор по умолчанию
- **AdjacencyList(vertices_count)** - конструктор с заданным размером
- **AdjacencyList(matrixGraph)** - конвертер из матричного графа

- **AddEdge(from, to, weight)** - добавление ребра
- **EraseRecord(from, to)** - удаление ребра
- **EraseVertex(vertex_number)** - удаление вершины
- **GetEdgesCount()** - получение количества рёбер
- **GetNeighbors(vertex)** - получение списка соседей
- **GetVerticesCount()** - получение количества вершин
- **HasEdge(from, to)** - проверка наличия ребра
- **GetEdgeWeight(from, to)** - получение веса ребра
- **Clear()** - очистка графа
- **IsEmpty()** - проверка на пустоту
- **MakeConnected(default_weight)** - достройка до связного
- **GenerateRandomConnectedGraph(vertices, density, maxWeight)**- генерация случайного графа
- **Dijkstra(start_vertex)** - алгоритм Дейкстры с бинарной кучей
- **DijkstraBinomial(start_vertex)** - алгоритм Дейкстры с биномиальной кучей
- **IsConnected()** - проверка связности
- **IsValidForDijkstra()** - проверка отсутствия отрицательных весов

Назначение: граф на списках смежности

Класс GraphApplication

Методы:

- **Run()** - запуск главного цикла программы
- **ClearGraphs()** - очистка графов
- **CreateGraph(size)** - создание пустого графа
- **GenerateRandomGraph()** - генерация случайного графа
- **AddEdge()** - добавление ребра (интерактивный)
- **RunDijkstra()** - запуск алгоритма Дейкстры
- **QueryShortestPath()** - запрос кратчайшего пути
- **PrintGraph()** - вывод графа на экран
- **ShowMenu()** - отображение главного меню

Назначение: управление пользовательским интерфейсом

3.3. Описание алгоритмов

Алгоритм Дейкстры (методы Dijkstra и DijkstraBinomial)

Описание: Находит кратчайшие расстояния от стартовой вершины до всех остальных вершин графа с неотрицательными весами. Алгоритм использует приоритетную очередь (бинарную или биномиальную кучу) для выбора вершины с минимальным расстоянием на каждом шаге.

Сложность по времени (где V – количество вершин, E – количество ребер):

- Для матрицы смежности + бинарная куча: $O(V^2 + E \log V)$
- Для матрицы смежности + биномиальная куча: $O(V^2 + E \log V)$
- Для списка смежности + бинарная куча: $O((V + E) \log V)$
- Для списка смежности + биномиальная куча: $O((V + E) \log V)$

Сложность по памяти: $O(V)$ для хранения расстояний, посещённых вершин и очереди

Бинарная куча (класс BinaryHeap)

Метод Insert

- **Описание:** Добавляет элемент в конец массива, затем выполняет просеивание вверх
- **Сложность по времени:** $O(\log n)$
- **Сложность по памяти:** $O(1)$

Метод ExtractMin

- **Описание:** Удаляет корневой элемент, перемещает последний элемент в корень, затем выполняет просеивание вниз
- **Сложность по времени:** $O(\log n)$
- **Сложность по памяти:** $O(1)$

Метод SiftUp

- **Описание:** Поднимает элемент вверх по дереву, пока он не займёт правильное место
- **Сложность по времени:** $O(\log n)$
- **Сложность по памяти:** $O(1)$

Метод SiftDown

- **Описание:** Опускает элемент вниз по дереву, пока он не займёт правильное место (больше детей)
- **Сложность по времени:** $O(\log n)$
- **Сложность по памяти:** $O(1)$

Биномиальная куча (класс BinomialHeap)

Метод Insert

- **Описание:** Создает кучу из одного узла и сливает её с текущей кучей
- **Сложность по времени:** $O(\log n)$
- **Сложность по памяти:** $O(1)$

Метод ExtractMin

- **Описание:** Находит узел с минимальным ключом, удаляет его, затем создаёт новую кучу из детей удалённого узла и сливает её с оставшейся частью
- **Сложность по времени:** $O(\log n)$
- **Сложность по памяти:** $O(1)$

Метод Merge

- **Описание:** Объединяет корневые списки двух куч, затем консолидирует деревья одинаковой степени (соединяет их, подвешивая дерево с большим корнем к дереву с меньшим)
- **Сложность по времени:** $O(\log n)$
- **Сложность по памяти:** $O(1)$

Метод DecreaseKey

- **Описание:** Уменьшает ключ узла и выполняет просеивание вверх
- **Сложность по времени:** $O(\log n)$
- **Сложность по памяти:** $O(1)$

Метод Delete

- **Описание:** Уменьшает ключ узла до минимального значения (минус бесконечность), затем извлекает минимальный элемент
- **Сложность по времени:** $O(\log n)$
- **Сложность по памяти:** $O(1)$

Метод IsConnected

Описание: Начиная с выбранной вершины, алгоритм проходит по всем смежным вершинам, пометая посещённые. Используется для проверки связности графа и нахождения компонент связности.

Сложность по времени: $O(V + E)$

Сложность по памяти: $O(V)$ для стека и массива посещённых вершин

Метод GenerateRandomConnectedGraph

Описание:

1. Создаётся пустой граф с заданным количеством вершин
2. Для каждой пары вершин (i, j) с вероятностью, равной плотности, добавляется ребро со случайным весом
3. Выполняется проверка связности с помощью DFS
4. Если граф несвязный, добавляются рёбра-мостики между компонентами связности

Сложность по времени: $O(V^2) + O(V + E)$ для генерации рёбер и для проверки связности

Сложность по памяти: $O(V^2)$ для матрицы смежности, $O(V + E)$ для списков смежности

Поиск компонент связности (метод MakeConnected)

Описание: Алгоритм находит все компоненты связности с помощью DFS. Затем между компонентами последовательно добавляются рёбра, чтобы превратить граф в связный. Достаточно добавить $(k-1)$ ребро, где k — количество компонент.

Сложность по времени: $O(V + E)$

Сложность по памяти: $O(V)$

Проверка графа на пригодность для алгоритма Дейкстры (метод IsValidForDijkstra)

Описание: Выполняет полный перебор всех рёбер графа и проверяет, что ни одно ребро не имеет отрицательного веса. При обнаружении отрицательного веса возвращает false.

Сложность по времени: $O(V^2)$ для матрицы смежности, $O(V + E)$ для списков смежности

Сложность по памяти: $O(1)$

4. Результаты экспериментов и тестирования

4.1. Описание экспериментов

В ходе работы были проведены экспериментальные исследования производительности четырех реализаций алгоритма Дейкстры:

Реализация 1: матрица смежности + бинарная куча

Реализация 2: матрица смежности + биномиальная куча

Реализация 3: списки смежности + бинарная куча

Реализация 4: списки смежности + биномиальная куча

Для экспериментов использовались четыре случайных связных графа со следующими характеристиками:

Граф	Число вершин	Плотность
№1	100	0.5
№2	250	0.75
№3	500	0.35
№4	1000	0.2

4.2. Результаты экспериментов:

Граф №1 на 100 вершинах

Номер	Тип графа	Тип кучи	Время выполнения(мс)
1	Матрица	Бинарная	8.70
2	Матрица	Биномиальная	6.27
3	Списки смежности	Бинарная	3.48
4	Списки смежности	Биномиальная	3.42

Граф №2 на 250 вершинах

Номер	Тип графа	Тип кучи	Время выполнения(мс)
1	Матрица	Бинарная	46.88
2	Матрица	Биномиальная	45.33
3	Списки смежности	Бинарная	50.61
4	Списки смежности	Биномиальная	30.70

Граф №3 на 500 вершинах

Номер	Тип графа	Тип кучи	Время выполнения(мс)
1	Матрица	Бинарная	89.28
2	Матрица	Биномиальная	117.21
3	Списки смежности	Бинарная	59.69
4	Списки смежности	Биномиальная	57.02

Граф №4 на 1000 вершин

Номер	Тип графа	Тип кучи	Время выполнения(мс)
1	Матрица	Бинарная	266.97
2	Матрица	Биномиальная	311.98
3	Списки смежности	Бинарная	163.21
4	Списки смежности	Биномиальная	152.10

4.3. Анализ результатов

Сравнение матрицы и списков смежности

Общая закономерность: На разреженных графах (плотность 0.2-0.35) списки смежности стабильно превосходят матрицу в 1.5-2 раза. На плотном графе (плотность 0.75) преимущество списков сохраняется только при использовании биномиальной кучи.

Сравнение бинарной и биномиальной куч

Матричное представление:

- Граф №1: биномиальная куча быстрее на 28% (6.27 мс против 8.70 мс)
- Граф №2: биномиальная куча быстрее на 3% (45.34 мс против 46.88 мс)
- Граф №3: бинарная куча быстрее на 31% (89.28 мс против 117.21 мс)
- Граф №4: бинарная куча быстрее на 17% (266.98 мс против 311.99 мс)

Списочное представление:

- Граф №1: разница незначительна (3.48 мс против 3.43 мс)
- Граф №2: биномиальная куча быстрее на 39% (30.71 мс против 50.61 мс)
- Граф №3: биномиальная куча быстрее на 4.5% (57.02 мс против 59.70 мс)
- Граф №4: биномиальная куча быстрее на 6.8% (152.10 мс против 163.22 мс)

Вывод по кучам: Преимущество биномиальной кучи наиболее заметно на плотных графах в паре со списками смежности (ускорение до 39%). На матричном представлении бинарная куча оказалась эффективнее на графах большого размера (500 и 1000 вершин).

4.4. Тестирование корректности

В ходе работы также были выполнены google-тесты для проверки корректности всех реализаций:

1. **Тест на фиксированном графе** — все четыре реализации дали одинаковые расстояния на графе из 4 вершин с известными кратчайшими путями
2. **Тест на случайном графе** — результаты алгоритмов с бинарной и биномиальной кучей совпали для всех вершин
3. **Тест на графе с одной вершиной** — алгоритм корректно возвращает нулевое расстояние
4. **Тесты куч** — проверены операции вставки, извлечения минимума и свойства упорядоченности для обеих реализаций куч

5. Тест на несвязном графе — алгоритм выбрасывает исключение

Все тесты успешно пройдены, что подтверждает корректность работы всех четырёх реализаций алгоритма Дейкстры.

Заключение

В ходе работы разработаны четыре реализации алгоритма Дейкстры, отличающиеся способом хранения графа (матрица или списки смежности) и типом приоритетной очереди (бинарная или биномиальная куча). Создано консольное приложение с возможностью выбора типа графа и кучи, генерации случайных графов и поиска кратчайших путей. Разработан набор google-тестов, подтверждающих корректность всех реализаций.

Проведённые экспериментальные исследования на графах размером от 100 до 1000 вершин показали, что комбинация списков смежности и биномиальной кучи стабильно является лучшей во всех тестах. Списки смежности превосходят матрицу на разреженных графах в 1.5–2.5 раза, а биномиальная куча в паре со списками даёт ускорение до 39% по сравнению с бинарной кучей. Матричное представление графа показало худшие результаты во всех экспериментах.

Таким образом, для практической реализации алгоритма Дейкстры рекомендуется использовать списки смежности в сочетании с биномиальной кучей — эта комбинация обеспечивает наилучшую производительность на графах различного размера и плотности.

Литература

Список источников, используемых при выполнении работы (если есть).

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: <https://en.cppreference.com/>